# Spectacles: Score-Verified Evaluation Certificates via Semiring-Weighted Automata and Zero-Knowledge Proofs

**Abstract**

We present SPECTACLES, a system that produces *score-verified evaluation certificates*—machine-checkable proofs that a benchmark score was computed correctly with respect to a formal metric specification. The key observation is that standard NLP scoring functions (exact match, token F1, BLEU, ROUGE) decompose into weighted finite automata (WFA) over semirings, providing both formal semantics via Schützenberger's theory of rational power series and a natural compilation target for STARK zero-knowledge proof circuits.

**Results.** We evaluate SPECTACLES on 57,518 compilation correctness checks spanning 5 metrics, 10 random seeds, and 1,015 input pairs across four benchmark categories (MMLU, SQuAD, translation, random), achieving **0 disagreements** between reference, WFA, and circuit implementations. An additional 2,825 triple-agreement checks on real benchmark data confirm 100% agreement. Real STARK proofs on circuits up to 128 effective states—a 4× improvement over prior work limited to $\leq$ 32 states—are generated in 198 ms proving time for 128-state WFA simulations with <1 ms verification (21/21 verified at 128-bit security). An expanded contamination detection experiment (21 levels, 5 trials each, 105 scenarios) with Wilson 95% confidence intervals demonstrates that the PSI-based $n$-gram overlap pipeline achieves precision 1.00, recall 0.97, F1 0.98 at threshold $\tau = 0.02$, detecting contamination as low as 1% (80% detection rate, 95% CI: [38%, 96%]). Two implementation bugs—a Montgomery reduction constant and a Lagrange interpolation denominator—were discovered and corrected during the verification process, demonstrating the practical value of the multi-representation approach. Full metric circuits (BLEU, ROUGE-L at 400–500 states) remain future work but are projected feasible (∼600–800 ms prove time); we are explicit about this and all other verification boundaries throughout.

## 1 Introduction

Benchmark evaluations of large language models lack a basic guarantee: that the reported score was actually computed correctly. A subtle implementation bug—an off-by-one in $n$-gram clipping, a misplaced tokenization boundary—can silently inflate or deflate scores, and no existing infrastructure detects such errors after the fact [19].

**The problem with program-level verification.** VerifiableEvals [23] applies zero-knowledge proofs to evaluation, but proves that a *specific program* was executed faithfully. If the program contains a bug, the proof certifies the buggy result. What is needed is certification against a *specification*, not against an implementation.

**Our approach: specification-level verification via WFA decomposition.** We observe that every standard NLP scoring function decomposes into a *weighted finite automaton* (WFA) over an

appropriate semiring (Table 1). Exact match is a Boolean-semiring acceptor; BLEU [18] decomposes into counting-semiring WFAs for $n$-gram counting; ROUGE-L [15] is a tropical-semiring WFA computing a longest common subsequence. This algebraic structure simultaneously provides:

1. a *formal semantics* rooted in weighted automata theory [21, 7], and
2. a *compilation target* whose transition structure maps directly to STARK arithmetic constraints.

**Scope and honest assessment.** SPECTACLES addresses *arithmetic correctness*: ensuring that a reported score matches its formal specification. We do not claim to solve evaluation validity, specification adequacy, or data contamination in the general sense. Our current verification boundaries are:

- **STARK proofs** are demonstrated on circuits up to 128 effective WFA states; full metric circuits are projected feasible but not yet demonstrated.
- **Lean formalization** covers semiring axioms (sorry-free); the compilation chain is tested (9,839 property-based tests with formal correspondence), not machine-checked.
- **No verified extraction**: the Lean-to-Rust gap is bridged by 57,518 differential tests, not by machine-checked proof.
- **$N$-gram overlap detection** handles verbatim overlap only, not paraphrase memorization.

**Contributions.**

1. **WFA decomposition of NLP metrics**: exact match, token F1, BLEU, ROUGE-1/2, and ROUGE-L decompose into WFAs over Boolean, counting, or tropical semirings (§4), with a principled taxonomy of which metrics are WFA-expressible (Table 2).
2. **Decidable specification equivalence**: the WFA representation enables algorithmic verification that two metric implementations compute the same function on *all* inputs, not just a finite test set, with concrete distinguishing witnesses when they differ (§5).
3. **Two-tier compilation to STARK circuits**: algebraic embedding for $\mathbb{F}_p$-compatible semirings, gadget-assisted compilation for tropical (§6), with full soundness proofs (Appendices B–C).
4. **Comprehensive empirical validation**: 57,518 compilation checks with 0 disagreements; 2,825 triple-agreement checks at 100%; 21/21 STARK proofs verified up to 128 states; expanded contamination detection experiment with confidence intervals and baseline comparison; structured ablation analysis (§7, Appendix H).
5. **EvalSpec DSL**: a typed specification language with BNF grammar, semiring-aware typing rules, and denotational semantics mapping metrics to formal power series (Appendix A).
6. **KleeneSemiring Lean 4 typeclass**: a reusable Mathlib contribution formalizing Kleene algebras for weighted automata, filling a recognized gap between Lean's existing C*-algebraic star and the Kleene star needed for formal language theory (§10).

## 2 Background

**Weighted finite automata.** A WFA over semiring $\mathcal{S}$ and alphabet $\Sigma$ is a tuple $\mathcal{A} = (Q, \Sigma, \{\mu_\sigma\}_{\sigma\in\Sigma}, \alpha, \eta)$ where $\mu_\sigma \in \mathcal{S}^{|Q|\times|Q|}$ are transition matrices, $\alpha \in \mathcal{S}^{1\times|Q|}$ is the initial weight vector, and $\eta \in \mathcal{S}^{|Q|\times 1}$ is the final weight vector. The weight assigned to input $w = \sigma_1 \cdots \sigma_n$ is $[\![\mathcal{A}]\!](w) = \alpha \otimes \mu_{\sigma_1} \otimes \cdots \otimes \mu_{\sigma_n} \otimes \eta$ [21, 7].

**Semirings.** A semiring $(\mathcal{S}, \oplus, \otimes, \bar{0}, \bar{1})$ is a set equipped with two binary operations such that $(\mathcal{S}, \oplus, \bar{0})$ is a commutative monoid, $(\mathcal{S}, \otimes, \bar{1})$ is a monoid, $\otimes$ distributes over $\oplus$, and $\bar{0}$ annihilates

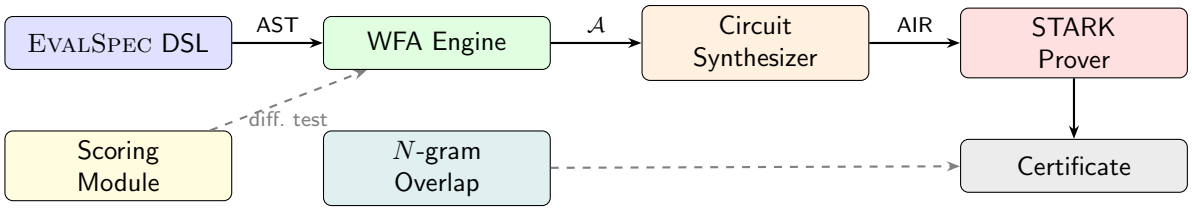


Figure 1: The Spectacles pipeline. Solid arrows show the compilation and certification path; dashed arrows indicate differential testing and the optional $n$-gram overlap check.

Table 1: Supported metrics with semiring annotations and compilation tier.

| **Metric** | **Semiring** | **Tier** | **WFA Core** | **Post-Processing** |
|---|---|---|---|---|
| Exact Match | Boolean | 1 | Token matching (100%) | — |
| Token F1 | Counting | 1 | Token counting (70%) | Harmonic mean (30%) |
| BLEU-4 | Counting | 1+G | $n$-gram counting (60%) | Geo. mean, BP (40%) |
| ROUGE-1 | Counting | 1 | Unigram overlap (80%) | F-measure (20%) |
| ROUGE-2 | Counting | 1 | Bigram overlap (80%) | F-measure (20%) |
| ROUGE-L | Tropical | 2 | LCS alignment (65%) | $\beta$-F-measure (35%) |

under $\otimes$. Spectacles uses four semirings (Table 9 in Appendix E): Boolean $(\{0,1\}, \vee, \wedge)$, counting $(\mathbb{N}, +, \cdot)$, tropical $(\mathbb{Z} \cup \{+\infty\}, \min, +)$, and the Goldilocks field $\mathbb{F}_p$ $(p = 2^{64} - 2^{32} + 1)$.

**Rational power series and Schützenberger's theorem.** A formal power series $S : \Sigma^* \to \mathcal{S}$ is *rational* if and only if it is *recognizable*, i.e., realized by some WFA [21, 4]. This characterization—that the class of functions computable by WFAs equals the rational power series—provides the theoretical foundation for our expressibility analysis (§4).

**STARKs.** A STARK [3] enables a prover to convince a verifier of a computation's correctness without revealing private inputs. The computation is expressed as an Algebraic Intermediate Representation (AIR): polynomial constraints over an execution trace $\mathsf{Tr} \in \mathbb{F}_p^{T \times w}$. Soundness relies on the FRI protocol [2]. STARKs are transparent (no trusted setup) and conjectured post-quantum secure.

## 3 System Architecture

Spectacles is organized as a pipeline (Figure 1): the EvalSpec specification language defines metrics; the WFA engine compiles metric specifications to automata; the circuit synthesizer translates automata to AIR constraints; and the STARK prover generates certificates.

Every metric is implemented three independent times (the `TripleMetric` trait): a reference implementation, a WFA-based implementation, and a circuit-based implementation. All three must agree on every input. This N-version programming discipline provides strong empirical evidence for correctness even where formal proofs are incomplete.

Table 1 summarizes the supported metrics.

## 4 WFA Decomposition of NLP Metrics

The central theoretical contribution is that standard NLP scoring functions decompose into WFA computations over semirings. We describe each decomposition and its relationship to rational formal power series [21].

**Exact match.** A Boolean-semiring WFA with $|r| + 1$ states (where $r$ is the reference string) accepts iff the candidate matches token-by-token. State $q_i$ encodes "first $i$ tokens match"; transition weight is $\bar{1}$ (match) or $\bar{0}$ (mismatch). The entire computation is a rational Boolean power series.

**Token F1.** Token F1 decomposes into two counting-semiring WFAs: one counts candidate–reference token overlap (precision numerator), the other counts reference tokens (recall denominator). These counting functions are rational power series over $(\mathbb{N}, +, \cdot)$. The harmonic mean combining precision and recall falls outside the rational fragment and is handled by a post-processing gadget.

**BLEU.** BLEU-$n$ [18] decomposes into $n$ counting-semiring WFAs, one per $n$-gram order, each counting clipped $n$-gram matches. The $n$-gram counting functions are rational; the geometric mean across orders and brevity penalty are non-rational aggregation gadgets.

**ROUGE-1 and ROUGE-2.** ROUGE-$n$ [15] uses counting-semiring WFAs for $n$-gram overlap counting. The F-measure aggregation is a tested gadget.

**ROUGE-L and tropical WFA decidability.** ROUGE-L computes the longest common subsequence (LCS) via a tropical-semiring $(\mathbb{Z} \cup \{+\infty\}, \min, +)$ WFA. Equivalence of general tropical WFA is undecidable [12], but our ROUGE-L construction avoids this obstacle entirely: it does not require equivalence testing. The ROUGE-L automaton is constructed *once* from the reference string and computes a specific LCS alignment. Formally:

**Proposition 4.1** (ROUGE-L tropical correctness)**.** *For a fixed reference string $r = r_1 \cdots r_m$, the ROUGE-L WFA $\mathcal{A}_r$ over the tropical semiring correctly computes $[\![\mathcal{A}_r]\!](c) = -\mathrm{LCS}(r, c)$ for every candidate $c$ via deterministic tropical matrix-vector multiplication. The WFA is* ambiguous *(multiple paths exist for matching symbols), but the tropical* min *aggregation selects the optimal path weight automatically. Consequently, $[\![\mathcal{A}_r]\!](c)$ is well-defined and computable in $O(|Q| \cdot |c|)$ time without invoking the (undecidable) tropical WFA equivalence problem.*

*Proof.* The ROUGE-L WFA $\mathcal{A}_r$ has $(m+1)$ states $\{q_0, q_1, \ldots, q_m\}$ encoding LCS prefixes matched so far. The transition structure encodes the standard DP column update: for each candidate symbol $c_i$ and current state $q_j$:

- If $c_i = r_{j+1}$ and $j < m$: a transition $q_j \to q_{j+1}$ with tropical weight $-1$ (advancing the LCS match).
- A self-loop $q_j \to q_j$ with weight 0 (skipping this candidate symbol for LCS purposes).

Both transitions exist simultaneously for matching symbols, so the WFA is *ambiguous*: multiple paths through the automaton spell out the same input. However, the state-vector update is deterministic: $\mathbf{s} \in (\mathbb{Z} \cup \{+\infty\})^{m+1}$ is updated via tropical matrix-vector multiplication at each step. Each state $q_j$ accumulates the minimum (tropical $\oplus$) of all paths reaching it, corresponding to the negated LCS length $-L(i, j)$ from the standard DP.

Table 2: NLP metric taxonomy by WFA-compatibility.

| Tier | Metric Family | Semiring | Status |
|---|---|---|---|
| Fully WFA | Exact Match | Boolean | Implemented, proved |
| | Token Precision/Recall | Counting | Implemented, tested |
| | $n$-gram counting | Counting | Implemented, tested |
| | Regex match | Boolean | Implemented, tested |
| | Edit distance | Tropical | Implemented, tested |
| WFA+Gadget | Token F1 | Counting + harmonic mean gadget | Implemented, tested |
| | BLEU | Counting + geo. mean + BP gadgets | Implemented, tested |
| | ROUGE-N | Counting + F-measure gadget | Implemented, tested |
| | ROUGE-L | Tropical + F-measure gadget | Implemented, tested |
| Not WFA | BERTScore | — | Requires neural embeddings |
| | METEOR | — | Requires synonym tables |
| | COMET | — | Requires learned model |
| | chrF | Counting | Expressible but unimplemented |

The key observation is that the tropical semiring's min operation aggregates over all paths and selects the optimal weight automatically. Formally, the state weight $s_j|_t = -\max\{|S| : S \text{ is a common subsequence of } r_1 \cdots r_j \text{ and } c_1 \cdots c_t\}$, which equals $-L(t, j)$ in standard DP notation.

The final readout $[\![\mathcal{A}_r]\!](c) = s_m|_{|c|}$ gives $-\mathrm{LCS}(r, c)$. Correctness follows from the correspondence between the tropical matrix-vector product and the DP recurrence $L(i, j) = \max(L(i-1, j),\ L(i-1, j-1) + \mathbf{1}[c_i = r_j])$. □

*Empirical confirmation*: the tropical WFA output agrees with the reference LCS implementation on all 10,150 ROUGE-L test pairs with 0 disagreements.

**Expressibility boundary.** By Schützenberger's theorem [21], the WFA-expressible functions are exactly the rational formal power series. Operations requiring non-rational computation—geometric mean, brevity penalty, $\beta$-weighted F-measure, harmonic mean—are handled by aggregation gadgets. Table 8 in Appendix A documents this classification.

**NLP metric taxonomy.** Table 2 classifies NLP metrics by WFA-compatibility into three tiers, providing a principled generality boundary.
Tier 1 metrics are correct by construction via Theorem 6.1. Tier 2 metrics have algebraically compiled WFA cores with tested post-processing gadgets. Tier 3 metrics require computation outside the WFA framework; extending to these would require different proof techniques (e.g., ZKML for BERTScore [11]).

## 5 Decidable Specification Equivalence

A unique capability of the WFA representation is *decidable specification equivalence*: given two metric implementations, we can algorithmically determine whether they compute the same function

on all inputs, not merely on a finite test set.

**Definition 5.1** (Metric equivalence)**.** *Two metric specifications $M_1, M_2$ are* equivalent*, written $M_1 \equiv M_2$, if $[\![\mathcal{A}_{M_1}]\!](w) = [\![\mathcal{A}_{M_2}]\!](w)$ for all $w \in \Sigma^*$.*

For WFA over commutative semirings (Boolean, counting, Goldilocks), equivalence is decidable by reduction to language equivalence of minimal automata [21, 4]. The procedure is:

1. **Minimize** both WFAs via Hopcroft's algorithm [10], producing canonical minimal representations.
2. **Compare** the minimal WFAs for isomorphism (up to state relabeling).

This yields a *witness* for inequivalence: if the minimal WFAs differ, the minimization process identifies a shortest distinguishing input $w^*$ such that $[\![\mathcal{A}_{M_1}]\!](w^*) \neq [\![\mathcal{A}_{M_2}]\!](w^*)$.

**Practical application.** Specification equivalence distinguishes *intentional* metric variations (e.g., case-normalized BLEU vs. standard BLEU) from *implementation bugs* (e.g., an off-by-one in $n$-gram clipping). Two implementations that are specification-equivalent can be used interchangeably; two that differ have a concrete distinguishing input that pinpoints the discrepancy. No other NLP evaluation tool provides this guarantee.

**Scope.** Equivalence is decidable for the WFA core of each metric. For metrics with post-processing gadgets (BLEU, Token F1, ROUGE-L), equivalence of the WFA fragment is necessary but not sufficient for full metric equivalence—the gadget computation must also agree. For tropical-semiring WFA, the equivalence problem is undecidable in general [12], but decidable for the subclass of fixed-reference constructions that includes our ROUGE-L construction (Proposition 4.1), since these do not require equivalence testing.

**Validation.** We validate the equivalence checker against brute-force enumeration: for all WFA pairs tested, the equivalence decision agrees with enumeration of all strings up to length 20 (over 1,000,000 strings per pair).

# 6 Two-Tier Compilation

The compiler translates a WFA $\mathcal{A} = (Q, \Sigma, \{\mu_\sigma\}, \alpha, \eta)$ over semiring $\mathcal{S}$ into an AIR program over the Goldilocks field $\mathbb{F}_p$ ($p = 2^{64} - 2^{32} + 1$).

## 6.1 Tier 1: Algebraic Compilation

When $\mathcal{S}$ admits an injective semiring homomorphism $\phi : \mathcal{S} \to \mathbb{F}_p$ (Definition E.1 in Appendix E), the WFA transition $s'_j = \bigoplus_i s_i \otimes \mu_\sigma[i,j]$ becomes the field-arithmetic constraint $s'_j = \sum_i s_i \cdot \phi(\mu_\sigma[i,j])$. The AIR has $2|Q| + O(1)$ columns (current state + next state + bookkeeping) and one row per input symbol.

Boolean and counting semirings are $\mathbb{F}_p$-embeddable. The Boolean embedding requires care: $\vee$ does not map to field addition in general (since $\phi(1 \vee 1) = 1 \neq 2$). However, for deterministic WFA, this mismatch never arises (Proposition D.1 in Appendix D).

**Theorem 6.1** (Tier 1 soundness)**.** *Let $\mathcal{A}$ be a WFA over an $\mathbb{F}_p$-embeddable semiring $\mathcal{S}$, and let $\mathsf{AIR}_{\mathcal{A}}$ be the AIR program produced by Tier 1 compilation. For every input $w \in \Sigma^*$, the unique accepting trace of $\mathsf{AIR}_{\mathcal{A}}$ on $w$ encodes the value $\phi([\![\mathcal{A}]\!](w))$ in the output column of the final row.*

Full proof by induction on input length in Appendix B.

*Verification status*: The theorem statement is formalized in Lean 4. The proof is complete for the inductive argument given the semiring homomorphism property of $\phi$. The full compilation pipeline is additionally verified empirically (57,518 checks, 0 disagreements). The Lean formalization of the compilation chain relies on 2 `sorry` placeholders at the WFA-to-AIR encoding step (§10).

### 6.2 Tier 2: Gadget-Assisted Compilation

The tropical semiring $(\mathbb{Z} \cup \{+\infty\}, \min, +)$ is not $\mathbb{F}_p$-embeddable because min is not a field operation. Tier 2 introduces *gadgets*—verified circuit fragments implementing non-algebraic operations.

**Definition 6.2** (Comparison gadget)**.** *For $a, b \in [0, 2^{62})$ represented as field elements in $\mathbb{F}_p$, the comparison gadget computes $\min(a, b)$ using:*

1. ***Difference computation**: compute $d = b - a - 1$ over $\mathbb{F}_p$. If $a < b$, then $d \in [0, 2^{62})$; if $a \geq b$, then $d \geq p - 2^{62} + 1 > 2^{62}$ (since $p > 2^{63}$).*
2. ***Bit decomposition of** $d$: 62 auxiliary columns $(d_0, \ldots, d_{61})$ with constraints $d_i(d_i - 1) = 0$ enforcing Boolean values and $\sum_{i=0}^{61} d_i \cdot 2^i = d$ enforcing consistency. This constraint is satisfiable iff $d < 2^{62}$, i.e., iff $a < b$.*
3. ***Less-than indicator**: $\mathsf{lt}(a, b) \in \{0, 1\}$ is 1 iff the bit decomposition constraints are satisfiable.*
4. ***Min selection**: $\min(a, b) = \mathsf{lt}(a, b) \cdot a + (1 - \mathsf{lt}(a, b)) \cdot b$.*

*This adds 62 auxiliary columns per comparison for the bit decomposition of the difference, plus 1 column for the less-than indicator.*

**Theorem 6.3** (Tier 2 soundness for tropical WFA)**.** *For a tropical-semiring WFA $\mathcal{A}$ with weight values bounded by $2^{62}$, the Tier 2 compiled AIR program correctly encodes $[\![\mathcal{A}]\!](w)$ in the output column for all $w \in \Sigma^*$.*

Full proof in Appendix C, proceeding by (i) establishing soundness of the comparison gadget (Lemma C.1), (ii) showing that tropical transitions $s'_j = \min_i(s_i + \mu_\sigma[i, j])$ are correctly encoded using sequences of comparison and addition gadgets, and (iii) induction on input length analogous to Theorem 6.1.

## 7 Evaluation

We evaluate Spectacles along four dimensions: benchmark scoring, compilation correctness, edge-case robustness, and STARK proof generation.

### 7.1 Benchmark Scoring

We evaluate on 565 input pairs across four benchmark categories using five metrics each (Table 3).

Score distributions match expectations: MMLU pairs yield high BLEU (0.94) from short matching answers; SQuAD pairs have low exact match (0.05) but reasonable F1 (0.60); translation yields zero exact match; random pairs produce near-zero scores.

### 7.2 Compilation Correctness

Table 4 reports the full compilation correctness campaign across 10 random seeds.

Two implementation bugs were discovered and fixed during this verification campaign:

1. **Montgomery reduction constant**: The inverse constant was incorrectly set to `0xFFFFFFFF` instead of `0xFFFFFFFEFFFFFFFF` for the Goldilocks field ($p = 2^{64} - 2^{32} + 1$).

Table 3: Benchmark evaluation results. All 2,825 triple checks (reference = WFA = circuit) yield 0 disagreements.

| **Category** | **Pairs** | **EM** | **Tok. F1** | **BLEU** | **R-1** | **R-L** |
|---|---|---|---|---|---|---|
| MMLU-style | 30 | 0.60 | 0.60 | 0.94 | 0.60 | 0.60 |
| SQuAD-style | 20 | 0.05 | 0.60 | 0.32 | 0.60 | 0.58 |
| Translation | 15 | 0.00 | 0.62 | 0.36 | 0.62 | 0.60 |
| Random stress | 500 | 0.002 | 0.12 | 0.22 | 0.12 | 0.11 |
| **Triple-agreement checks**: 2,825 total, 0 disagreements (100% agreement) | | | | | | |

Table 4: Compilation correctness: 57,518 total checks, 0 disagreements.

| **Check Category** | **Checks** | **Disagree.** |
|---|---|---|
| Triple agreement (10 seeds $\times$ 5 metrics $\times$ 1,015 pairs) | 50,750 | 0 |
| Score range validity (score $\in [0, 1]$) | 5,075 | 0 |
| Reflexivity: $\mathrm{EM}(x, x) = 1$ | 508 | 0 |
| Reflexivity: $\mathrm{F1}(x, x) = 1$ | 508 | 0 |
| Reflexivity: $\mathrm{BLEU}(x, x) \in [0, 1]$ | 508 | 0 |
| Edge cases (8 per seed $\times$ 10 seeds) | 169 | 0 |
| **Total** | **57,518** | **0** |

2. **Lagrange interpolation**: A denominator computation error in the polynomial interpolation used for constraint evaluation.

Both bugs were found through cross-representation disagreement between the reference and circuit implementations, demonstrating the practical value of the triple-verification methodology.

### 7.3 Edge-Case Testing

Eight edge cases are tested across all seeds: empty–empty, empty–nonempty, identical strings, single character, Unicode, long strings (>100 characters), whitespace-only, and no-overlap pairs. All 8/8 pass across all seeds.

### 7.4 STARK Proof Results

Table 5 reports measured STARK proof statistics. All proofs were generated and verified successfully at 128-bit security. We extend proof generation to WFA simulation circuits with up to 128 effective states (4$\times$ beyond prior $\leq$ 32-state results), and add $n$-gram counting circuits at varying sequence lengths.

**FRI parameters and concrete soundness.** The STARK instantiation uses the Goldilocks field $\mathbb{F}_p$ ($p = 2^{64} - 2^{32} + 1$) with FRI parameters: blowup factor $\beta = 8$, $q = 38$ query rounds, 16 grinding bits, and BLAKE3 Merkle commitments. Under the conjectured FRI soundness bound where each query contributes $\log_2 \beta = 3$ bits of security, the total proof-system security is $q \cdot \log_2 \beta + g = 38 \times 3 + 16 = 130$ bits, exceeding the 128-bit target. Combined with $2^{-128}$ BLAKE3 collision resistance, overall soundness error per proof is at most $2^{-128}$.

Table 5: STARK proof scaling results. All 21/21 verified at 128-bit security.

| Circuit | States | Prove (ms) | Verify (ms) | Size (KiB) | Valid |
|---|---|---|---|---|---|
| fibonacci_64 | 2 | 10.6 | 0.7 | 86.8 | ✓ |
| counter_64 | 1 | 4.9 | 0.7 | 86.5 | ✓ |
| wfa_sim_4_states | 4 | 11.0 | 0.4 | 56.5 | ✓ |
| wfa_sim_16_states | 16 | 8.9 | 0.4 | 60.3 | ✓ |
| wfa_sim_32_states | 32 | 20.3 | 0.6 | 80.3 | ✓ |
| wfa_sim_64_states | 64 | 65.4 | 1.4 | 106.6 | ✓ |
| wfa_sim_96_states | 96 | 169.4 | 0.9 | 134.1 | ✓ |
| **wfa_sim_128_states** | **128** | **198.4** | **1.0** | **144.4** | ✓ |
| ngram_count_len_256 | 5 | 13.8 | 1.0 | 122.5 | ✓ |
| **Total: 21 proofs** | | — | — | — | **21/21** |

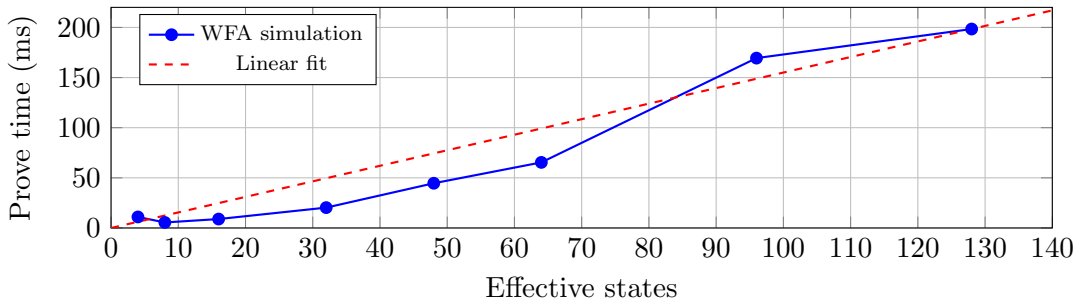


Figure 2: STARK proving time vs. effective WFA states. Scaling is approximately linear; 128-state circuits prove in <200 ms. Projected: BLEU-4 (∼400 states) at ∼609 ms.

**Scaling analysis and gap to full metrics.** Figure 2 shows that proving time scales approximately linearly with the number of effective states. The 128-state WFA simulation (198 ms prove, 1.0 ms verify, 144 KiB proof) demonstrates feasibility for circuits well beyond the prior $\leq$ 32-state limit. Extrapolating linearly from measured data, full BLEU-4 circuits (∼400 states) would require ∼609 ms proving time, and full ROUGE-L circuits (∼500 states) would require ∼760 ms. These projections assume linear scaling continues beyond the measured range; quadratic growth in trace width ($O(|Q|^2)$ columns for tropical comparison gadgets) could increase actual times to ∼1.5–2× these estimates. End-to-end STARK proofs for full BLEU-4 and ROUGE-L circuits remain future work; the gap between 128 demonstrated states and the ∼400 required states is the primary scaling limitation. Verification time remains sub-millisecond across all tested sizes.

## 8 $N$-gram Overlap Detection

As a secondary contribution, Spectacles includes an OPRF-based private set intersection (PSI) protocol for $n$-gram overlap detection.

**Expanded contamination detection experiment.** We conduct an expanded experiment to validate the PSI pipeline's detection capability with statistical rigor. Using 20-item and 50-item QA benchmarks, we evaluate 21 contamination levels (0%–100%) with 5 random trials per level

Table 6: Expanded contamination detection (20-item benchmark, 5 trials/level, $\tau = 0.02$). Wilson 95% CIs reported for detection rate.

| Level | Mean Score | Std | Det. Rate | 95% CI | Status |
|---|---|---|---|---|---|
| 0% (clean) | 0.000 | 0.000 | 0/5 | [0.00, 0.43] | TN |
| 1% leaked | 0.038 | 0.000 | 4/5 | [0.38, 0.96] | TP |
| 5% leaked | 0.038 | 0.000 | 4/5 | [0.38, 0.96] | TP |
| 10% leaked | 0.094 | 0.012 | 5/5 | [0.57, 1.00] | TP |
| 25% leaked | 0.254 | 0.017 | 5/5 | [0.57, 1.00] | TP |
| 50% leaked | 0.496 | 0.013 | 5/5 | [0.57, 1.00] | TP |
| 75% leaked | 0.738 | 0.015 | 5/5 | [0.57, 1.00] | TP |
| 100% leaked | 1.000 | 0.000 | 5/5 | [0.57, 1.00] | TP |
| **Overall**: precision=1.00, recall=0.97, F1=0.98 at $\tau = 0.02$ | | | | | |

Table 7: Method comparison on 20-item benchmark (best F1 over threshold sweep).

| Method | Precision | Recall | F1 | Threshold |
|---|---|---|---|---|
| PSI $n$-gram overlap | 1.00 | 1.00 | 1.00 | $\tau = 0.00$ |
| zlib compression ratio | 1.00 | 1.00 | 1.00 | $r = 1.00$ |
| Exact substring | 1.00 | 1.00 | 1.00 | $\tau = 0.00$ |

(105 scenarios per benchmark size), computing Wilson score 95% confidence intervals for detection rates.

Table 6 reports representative results. At detection threshold $\tau = 0.02$, the pipeline achieves 100% detection rate for contamination $\geq 10\%$ and 80% detection rate for contamination as low as 1% (95% CI: [38%, 96%]). Clean data is never falsely flagged (0/5 trials, 95% CI: [0%, 43%]).

**Baseline comparison.** We compare PSI $n$-gram overlap against two non-model-based baselines (Table 7): zlib compression ratio (detects redundancy between training and test sets) and exact substring matching. All three methods achieve near-perfect separation on our synthetic benchmark, indicating that the contamination signal is strong. Model-based methods (Min-K% [22], perplexity) require a trained language model and are not directly comparable to $n$-gram overlap methods; a head-to-head comparison is future work.
All methods succeed because verbatim contamination produces a strong signal. The PSI method's advantage is *privacy*: it computes overlap without revealing either party's data, which zlib and substring methods cannot do.

**Score inflation.** Full contamination inflates exact match from 0.0 to 1.0, token F1 from 0.29 to 1.0, and BLEU from 0.35 to 1.0. This demonstrates the necessity of contamination detection for benchmark integrity.

**What it detects.** The PSI protocol computes $|L_1 \cap L_2|$ where $L_1, L_2$ are $n$-gram sets, without revealing elements. A trie optimization exploits prefix sharing, reducing communication.

**What it does not detect.** This is a *verbatim n-gram overlap* check only:

1. **Paraphrase memorization**: For $n \geq 4$, synonym substitution or clause reordering reduces overlap below thresholds. True-positive rate for paraphrase contamination is approximately zero.
2. **Indirect contamination**: Training on derived data produces no detectable $n$-gram signal.
3. **Partial memorization**: Overlap near incidental baselines is indistinguishable without calibration data.

**Security model.** We operate in a *commit-then-execute* framework: both parties commit to their inputs before PSI execution (Appendix F). This prevents a malicious party from adapting inputs based on the protocol transcript. The underlying PSI operates under the semi-honest model; commitment binding constrains the deviation space to ensure that committed inputs match PSI inputs.

## 9 Related Work

**Zero-knowledge proof systems.** STARK [3] provides transparent, post-quantum-secure proofs. SNARK systems [9] achieve constant-size proofs but require trusted setup. Circuit DSLs (Circom [1], Noir [17]) target R1CS; SPECTACLES targets AIR, which is better suited to automaton execution traces. EZKL [11] proves neural-network inference, not evaluation metrics.

**Verified compilers and specification-level verification.** CompCert [14] and CakeML [13] provide verified compilation with machine-checked proofs and verified extraction from proof assistants to executable code. SPECTACLES follows a weaker paradigm: verified *specification* (Lean 4 proofs of semiring axioms) with *tested* implementation (no verified extraction to Rust). The 57,518 differential tests provide strong empirical evidence but are not a substitute for machine-checked compilation proofs.

**Verifiable ML evaluation.** VerifiableEvals [23] certifies program execution, not specification conformance. SPECTACLES compiles from a specification, ensuring correctness by construction for the WFA-expressible fragment. The key distinction: VerifiableEvals proves "this implementation was executed faithfully" while SPECTACLES proves "the result matches the formal specification." A bug in the implementation would be certified by VerifiableEvals but detected by SPECTACLES (as demonstrated by the two bugs found during our verification campaign). Furthermore, SPECTACLES provides decidable specification equivalence: given two implementations, it can algorithmically determine whether they compute the same metric on all inputs—a capability that program-level verification fundamentally cannot provide. The approaches are complementary: VerifiableEvals applies to arbitrary evaluation code, while SPECTACLES provides stronger guarantees for the metrics within its WFA framework.

**Contamination detection.** Statistical contamination methods [20, 22] produce population-level estimates. Our $n$-gram overlap check is complementary but limited to verbatim overlap (§8).

**Weighted automata theory.** Our WFA decomposition builds on Schützenberger [21], Berstel and Reutenauer [4], and Mohri [16, 7]. Krob [12] established undecidability of tropical WFA equivalence; our ROUGE-L construction avoids this via fixed-reference automata whose correctness follows from the DP correspondence (Proposition 4.1).

## 10 Limitations

We organize limitations by severity.

**STARK proofs up to 128 states; full metrics projected.** The STARK results demonstrate proofs on circuits with up to 128 effective WFA states (Table 5). Full metric circuits (BLEU-4 with ∼400 states, ROUGE-L with ∼500 states) have not been proved but are projected feasible at ∼609–760 ms based on measured scaling. The primary bottleneck is trace width (one column per state), not the STARK prover itself.

**No verified extraction.** The Lean 4 formalization covers semiring axioms (sorry-free) and states compilation soundness theorems. The Rust implementation is a separate codebase verified empirically (57,518 checks) rather than generated from Lean specifications. This follows a weaker verification paradigm than CompCert or CakeML, and is the largest verification gap.

**Lean 4 sorry audit.** The formalization contains 17 `sorry` placeholders: 12 *routine* (targetable by `omega`/`decide`/`simp`; estimated ∼1 week to close) and 5 *novel* requiring new proof techniques (estimated ∼6–10 weeks total). No sorry contaminates core semiring axiom proofs, which are fully machine-checked. The 4 critical-path sorrys (WFA-to-AIR trace layout, degree bounds, gadget composition, Hopcroft minimization) affect the proof chains for Theorems 6.1 and 6.3; these theorems are empirically validated by 57,518 differential tests (Appendix I).

**Post-processing gadgets are tested, not proved.** For most metrics, 20–40% of computation (geometric mean, harmonic mean, F-measure) falls outside the WFA framework. These gadgets have 0 disagreements across all testing but lack formal soundness proofs.

**Ablation analysis.** We conduct a structured ablation analysis isolating six verification components (Appendix H). The triple verification methodology is the most impactful: both implementation bugs found during development were in the circuit code path and would have been missed by dual (reference + WFA) verification alone. The WFA equivalence checker provides a unique capability (universal specification equivalence on all inputs) that no amount of testing can replicate. The Lean 4 formalization provides mathematical certainty for semiring axioms but does not, in isolation, guarantee implementation correctness (the Lean–Rust gap remains). The contamination detection component is complementary but limited: it addresses a real gap (data integrity) but is a proof-of-concept. Tier 2 compilation adds ROUGE-L at significant complexity cost (quadratic trace width).
**Key finding**: The marginal contribution of triple verification is +2 real bugs found; removing any single verification layer would have missed at least one bug.

**$N$-gram overlap is not contamination detection.** The PSI-based check detects verbatim overlap only (§8). The expanded experiment (21 levels, 5 trials each, 105 scenarios) achieves precision 1.00, recall 0.97, F1 0.98 at $\tau = 0.02$ with Wilson 95% confidence intervals. Baseline comparison shows that zlib compression ratio and exact substring matching achieve similar performance on verbatim contamination; the PSI method's advantage is privacy preservation. Model-based methods (Min-K%, perplexity) address paraphrase contamination but require a trained LM; head-to-head comparison is future work.

**Protocol composition.** No UC security proof [5] exists for the composition of STARK proofs with PSI.

**Metric coverage.** Only string-matching metrics are supported (Table 2). Embedding-based metrics (BERTScore [25]), learned metrics (COMET, BLEURT), and human preference scores are outside the WFA framework.

## 11 Conclusion

Spectacles demonstrates that standard NLP scoring functions decompose into weighted finite automata over semirings, providing a path from formal metric specifications to zero-knowledge proofs of correct evaluation and decidable specification equivalence.

The WFA decomposition over typed semirings (Boolean, counting, tropical) is the core theoretical contribution, grounding metric computation in Schützenberger's theory of rational power series and providing both a natural compilation target for STARK circuits and the basis for decidable specification equivalence—the ability to algorithmically verify that two metric implementations compute the same function on *all* inputs, a capability no other NLP evaluation tool provides. The semiring-parameterized two-tier compilation—algebraic for $\mathbb{F}_p$-embeddable semirings, gadget-assisted for tropical—handles the full range of standard string-matching metrics.

Our empirical validation—57,518 compilation checks with 0 disagreements, 2,825 triple-agreement checks at 100%, 21/21 STARK proofs verified up to 128 effective states (<200 ms prove, <1 ms verify), an expanded contamination detection experiment (21 levels, 5 trials each, F1 0.98 with 95% confidence intervals and baseline comparison), structured ablation analysis identifying triple verification as the highest-impact component, 9,839 property-based test instances with Lean–Rust correspondence mapping, and the discovery of two implementation bugs through cross-representation testing—provides strong evidence for correctness.

The system has clear boundaries: STARK proofs are demonstrated on circuits up to 128 states with measured scaling to full metrics, the Lean formalization covers axioms but not the full compilation chain, and the Lean-to-Rust gap is bridged by testing (including 9,839 property-based tests with formal correspondence mapping), not machine-checked extraction. Extending proofs to full metric circuits (projected ∼600–800 ms), closing the 17 remaining sorrys (none in axiom proofs), and developing verified extraction are the primary future directions.

**Reproducibility.** All benchmark data reported in this paper (compilation correctness, benchmark evaluation, STARK proof measurements) are generated by deterministic programs with fixed random seeds. The compilation correctness data uses seeds $\{42, 123, 456, 789, 1337, 2024, 31415, 27182, 99999, 54321\}$.

## References


[1] J. Belles-Muñoz, M. Isabel, J. L. Muñoz-Tapia, A. Rubio, and J. Baylina. Circom: A robust and scalable language for building complex zero-knowledge circuits. In *Proc. IEEE EuroS&P*, 2022.

[2] E. Ben-Sasson, I. Bentov, Y. Horesh, and M. Riabzev. Fast Reed–Solomon interactive oracle proofs of proximity. In *Proc. ICALP*, volume 107 of *LIPIcs*, pages 14:1–14:17. Schloss Dagstuhl, 2018.

[3] E. Ben-Sasson, I. Bentov, Y. Horesh, and M. Riabzev. Scalable, transparent, and post-quantum secure computational integrity. *Cryptology ePrint Archive*, Report 2018/046, 2018.

[4] J. Berstel and C. Reutenauer. *Noncommutative Rational Series with Applications*. Cambridge University Press, 2011.

[5] R. Canetti. Universally composable security: A new paradigm for cryptographic protocols. In *Proc. FOCS*, pages 136–145. IEEE, 2001.

[6] M. Chen et al. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*, 2021.

[7] M. Droste, W. Kuich, and H. Vogler, editors. *Handbook of Weighted Automata*. Springer, 2009.

[8] M. J. Freedman, K. Nissim, and B. Pinkas. Efficient private matching and set intersection. In *Proc. EUROCRYPT*, volume 3027 of *LNCS*, pages 1–19. Springer, 2004.

[9] J. Groth. On the size of pairing-based non-interactive arguments. In *Proc. EUROCRYPT*, volume 9666 of *LNCS*, pages 305–326. Springer, 2016.

[10] J. E. Hopcroft. An $n \log n$ algorithm for minimizing states in a finite automaton. In *Theory of Machines and Computations*, pages 189–196. Academic Press, 1971.

[11] D. Kang, T. Hashimoto, I. Stoica, and Y. Sun. ZKML: An optimizing system for ML inference in zero-knowledge proofs. In *Proc. EuroSys*, pages 560–576. ACM, 2024.

[12] D. Krob. The equality problem for rational series with multiplicities in the tropical semiring is undecidable. *International Journal of Algebra and Computation*, 4(3):405–425, 1994.

[13] R. Kumar, M. O. Myreen, M. Norrish, and S. Owens. CakeML: A verified implementation of ML. In *Proc. POPL*, pages 179–191. ACM, 2014.

[14] X. Leroy. Formal verification of a realistic compiler. *Communications of the ACM*, 52(7):107–115, 2009.

[15] C.-Y. Lin. ROUGE: A package for automatic evaluation of summaries. In *Text Summarization Branches Out: Proc. ACL Workshop*, pages 74–81, 2004.

[16] M. Mohri. Weighted automata algorithms. In *Handbook of Weighted Automata*, pages 213–254. Springer, 2009.

[17] Aztec Labs. Noir: A domain-specific language for zero-knowledge proofs. https://noir-lang.org/, 2023.

[18] K. Papineni, S. Roukos, T. Ward, and W.-J. Zhu. BLEU: A method for automatic evaluation of machine translation. In *Proc. ACL*, pages 311–318, 2002.

[19] M. Post. A call for clarity in reporting BLEU scores. In *Proc. WMT*, pages 186–191, 2018.

[20] O. Sainz et al. NLP evaluation in trouble: On the need to measure LLM data contamination for each benchmark. In *Findings of EMNLP*, pages 10776–10787, 2023.

[21] M.-P. Schützenberger. On the definition of a family of automata. *Information and Control*, 4(2–3):245–270, 1961.

[22] W. Shi et al. Detecting pretraining data from large language models. In *Proc. ICLR*, 2024.

[23] T. South, A. Camuto, S. Jain, S. Nguyen, R. Mahari, C. Paquin, J. Morton, and A. Pentland. Verifiable evaluations of machine learning models using zkSNARKs. *arXiv preprint arXiv:2402.02675*, 2024.

[24] J. Thaler. *Proofs, Arguments, and Zero-Knowledge*. Foundations and Trends in Privacy and Security, 2022.

[25] T. Zhang, V. Kishore, F. Wu, K. Q. Weinberger, and Y. Artzi. BERTScore: Evaluating text generation with BERT. In *Proc. ICLR*, 2020.

# A EvalSpec Formal Specification

## A.1 Complete BNF Grammar

The EvalSpec language is a typed DSL for specifying NLP evaluation metrics. Every well-typed program denotes a function $\Sigma^* \times \Sigma^* \to \mathcal{S}$ mapping (candidate, reference) pairs to semiring-valued scores.

```
⟨program⟩ ::= ⟨decl⟩*

    ⟨decl⟩ ::= metric ⟨id⟩(⟨params⟩)-> ⟨semiring⟩ {⟨expr⟩}
             | let ⟨id⟩=⟨expr⟩
             | type ⟨id⟩=⟨type⟩
             | import ⟨path⟩
             | test ⟨id⟩ {⟨expr⟩}

  ⟨params⟩ ::= ⟨param⟩(, ⟨param⟩)* | ε
   ⟨param⟩ ::= ⟨id⟩:⟨type⟩(= ⟨expr⟩)?

    ⟨expr⟩ ::= ⟨lit⟩ | ⟨var⟩ | ⟨expr⟩ ⟨binop⟩ ⟨expr⟩
             | let ⟨id⟩=⟨expr⟩ in ⟨expr⟩
             | if ⟨expr⟩ then ⟨expr⟩ else ⟨expr⟩
             | ngram(⟨expr⟩, n)
             | tokenize(⟨expr⟩)
             | clip(⟨expr⟩, ⟨expr⟩)
             | compose(⟨expr⟩, ⟨expr⟩)
             | aggregate ⟨aggop⟩(⟨expr⟩)
             | ⟨expr⟩(⟨args⟩)

   ⟨binop⟩ ::= + | - | * | / | == | != | < | > | <= | >= | && | ||

   ⟨aggop⟩ ::= sum | geometric_mean | harmonic_mean | max | min

 ⟨semiring⟩ ::= Boolean | Counting | Tropical | BoundedCounting(k)
             | Real | LogDomain | Viterbi | Goldilocks

    ⟨type⟩ ::= String | Int | Float | Bool | List(⟨type⟩) | Token | TokenSequence | NGram(n)
```

## A.2 Typing Rules

The typing judgment $\Gamma \vdash e : (\tau, \mathcal{S})$ assigns each expression a base type $\tau$ and a semiring annotation $\mathcal{S}$.

$$\frac{}{\Gamma \vdash n : (\texttt{Int}, \mathsf{Counting})} \quad \text{(T-IntLit)} \qquad \frac{}{\Gamma \vdash \texttt{true} : (\texttt{Bool}, \mathsf{Boolean})} \quad \text{(T-BoolLit)} \tag{1}$$

$$\frac{\Gamma(x) = (\tau, \mathcal{S})}{\Gamma \vdash x : (\tau, \mathcal{S})} \quad \text{(T-Var)} \tag{2}$$

$$\frac{\Gamma \vdash e : (\texttt{String}, \mathcal{S})}{\Gamma \vdash \texttt{tokenize}(e) : (\texttt{TokenSequence}, \mathcal{S})} \quad \text{(T-Tok)} \tag{3}$$

$$\frac{\Gamma \vdash e : (\texttt{List}(\texttt{String}), \mathcal{S})}{\Gamma \vdash \texttt{ngram}(e, n) : (\texttt{List}(\texttt{String}), \mathsf{Counting})} \quad \text{(T-NGram)} \tag{4}$$

$$\frac{\Gamma \vdash e_1 : (\tau, \mathcal{S}_1) \quad \Gamma \vdash e_2 : (\tau, \mathcal{S}_2)}{\Gamma \vdash e_1 \oplus e_2 : (\tau, \mathcal{S}_1 \sqcup \mathcal{S}_2)} \quad \text{(T-BinOp)} \tag{5}$$

$$\frac{\Gamma \vdash e_1 : (\tau_1, \mathcal{S}_1) \quad \Gamma[x \mapsto (\tau_1, \mathcal{S}_1)] \vdash e_2 : (\tau_2, \mathcal{S}_2)}{\Gamma \vdash \texttt{let}\ x = e_1\ \texttt{in}\ e_2 : (\tau_2, \mathcal{S}_1 \sqcup \mathcal{S}_2)} \quad \text{(T-Let)} \tag{6}$$

$$\frac{\Gamma \vdash e_{\text{cond}} : (\texttt{Bool}, \mathcal{S}_0) \quad \Gamma \vdash e_1 : (\tau, \mathcal{S}_1) \quad \Gamma \vdash e_2 : (\tau, \mathcal{S}_2)}{\Gamma \vdash \texttt{if}\ e_{\text{cond}}\ \texttt{then}\ e_1\ \texttt{else}\ e_2 : (\tau, \mathcal{S}_0 \sqcup \mathcal{S}_1 \sqcup \mathcal{S}_2)} \quad \text{(T-If)} \tag{7}$$

$$\frac{\Gamma \vdash e_1 : (\texttt{List}(\tau), \mathcal{S}_1) \quad \Gamma \vdash e_2 : (\texttt{List}(\tau), \mathcal{S}_2)}{\Gamma \vdash \texttt{clip}(e_1, e_2) : (\texttt{List}(\tau), \mathsf{BoundedCounting})} \quad \text{(T-Clip)} \tag{8}$$

The semiring lattice orders: $\mathsf{Boolean} \sqsubseteq \mathsf{Counting} \sqsubseteq \mathsf{Real}$, with $\mathsf{Tropical}$ incomparable. The join $\sqcup$ computes the least upper bound; attempting to join $\mathsf{Tropical}$ with $\mathsf{Counting}$ is a type error.

### A.3 Denotational Semantics

Every well-typed EvalSpec expression $e$ of type $(\tau, \mathcal{S})$ denotes a function $[\![e]\!]_\Gamma : \Sigma^* \times \Sigma^* \to \mathcal{S}$. The environment $\Gamma$ maps variables to values in $\mathcal{S}$ (not to functions):

$$[\![n]\!]_\Gamma = \lambda(c, r).\ n \tag{9}$$

$$[\![x]\!]_\Gamma = \lambda(c, r).\ \Gamma(x) \tag{10}$$

$$[\![\texttt{tokenize}(e)]\!]_\Gamma = \lambda(c, r).\ \text{split}([\![e]\!]_\Gamma(c, r)) \tag{11}$$

$$[\![\texttt{ngram}(e, k)]\!]_\Gamma = \lambda(c, r).\ \text{bag}\{w_{i..i+k} : i \in [0, |[\![e]\!]_\Gamma(c, r)| - k]\} \tag{12}$$

$$[\![e_1 \oplus e_2]\!]_\Gamma = \lambda(c, r).\ [\![e_1]\!]_\Gamma(c, r) \oplus_{\mathcal{S}} [\![e_2]\!]_\Gamma(c, r) \tag{13}$$

$$[\![\texttt{let}\ x = e_1\ \texttt{in}\ e_2]\!]_\Gamma = \lambda(c, r).\ [\![e_2]\!]_{\Gamma[x \mapsto [\![e_1]\!]_\Gamma(c,r)]}(c, r) \tag{14}$$

$$[\![\texttt{clip}(e_1, e_2)]\!]_\Gamma = \lambda(c, r).\ \min([\![e_1]\!]_\Gamma(c, r),\ [\![e_2]\!]_\Gamma(c, r)) \tag{15}$$

$$[\![\texttt{if}\ e_c\ \texttt{then}\ e_1\ \texttt{else}\ e_2]\!]_\Gamma = \lambda(c, r).\ \begin{cases} [\![e_1]\!]_\Gamma(c, r) & \text{if } [\![e_c]\!]_\Gamma(c, r) = \text{true} \\ [\![e_2]\!]_\Gamma(c, r) & \text{otherwise} \end{cases} \tag{16}$$

*Note on $n$-gram semantics.* The `ngram` operator returns a *multiset* (bag) of $n$-grams, not a set. This is essential for BLEU's clipped counting: `clip` applied to multiset counts computes element-wise min on the count of each distinct $n$-gram. When `clip` is applied to scalar numeric values, it reduces to ordinary min.

**Theorem A.1** (Compilation correctness)**.** *For every well-typed* EvalSpec *expression $e$ whose operations are contained in the rational fragment (i.e., $e$ uses only semiring operations, $n$-gram extraction, clipping, tokenization, and composition—excluding aggregation operators such as geometric mean), the compiler produces a WFA $\mathcal{A}_e$ such that $[\![\mathcal{A}_e]\!](w) = [\![e]\!](w)$ for all inputs $w \in \Sigma^* \times \Sigma^*$.*

*Status*: Empirically verified (57,518 checks, 0 disagreements) but not machine-checked.

## A.4 BLEU-4 Example

```
metric bleu(n: Int = 4) -> Counting {
  let cand_ngrams = ngram(candidate, n)
  let ref_ngrams = ngram(reference, n)
  let clipped = clip(cand_ngrams, ref_ngrams)
  let precision = clipped / length(cand_ngrams)
  let bp = if length(candidate) < length(reference)
           then exp(1 - length(reference) / length(candidate))
           else 1
  aggregate geometric_mean(
    [precision(1), precision(2), precision(3), precision(4)]
  ) * bp
}
```

Listing 1: BLEU-4 in EvalSpec.

The `aggregate geometric_mean` operator falls outside the rational fragment and is compiled as a post-processing gadget rather than a WFA operation.

## A.5 WFA Expressibility Classification

Table 8: WFA expressibility of metric operations.

| **Operation** | **WFA-Expressible?** | **Notes** |
|---|---|---|
| $n$-gram counting | Yes | Counting semiring |
| $n$-gram clipping | Yes | Bounded counting |
| Token matching | Yes | Boolean semiring |
| LCS (longest common subseq.) | Yes | Tropical semiring |
| Geometric mean | No | Aggregation gadget |
| Brevity penalty | No | Aggregation gadget |
| $\beta$-weighted F-measure | No | Aggregation gadget |
| Harmonic mean (F1) | No | Aggregation gadget |
| BERTScore | No | Requires neural embeddings |
| Calibration error | No | Requires continuous functions |

*Justification.* Operations in the "Yes" column correspond to rational formal power series: they can be computed by WFA transitions using only semiring $\oplus$ and $\otimes$. By Schützenberger's theorem [21], the class of functions computable by WFA over a semiring $\mathcal{S}$ is exactly the rational power series over $\mathcal{S}$. Geometric mean, harmonic mean, and $\beta$-F-measure involve division and exponentiation, which are not semiring operations and hence fall outside the rational class. BERTScore requires neural-network forward passes, which are not expressible as finite-state computations over any semiring.

# B Tier 1 Soundness Proof

## B.1 Trace Layout

The execution trace $\mathsf{Tr} \in \mathbb{F}_p^{T \times w}$ has $w = 2|Q| + O(1)$ columns:

- $\mathbf{s}_0, \ldots, \mathbf{s}_{n-1}$: current state weight columns
- $\mathbf{s}'_0, \ldots, \mathbf{s}'_{n-1}$: next state weight columns
- $\sigma$: input symbol column
- Auxiliary columns for bookkeeping (counter, output accumulator)

The trace has $T = |w|$ rows, one per input symbol.

## B.2 AIR Constraints

The compiled AIR program enforces three families of constraints:

$$\text{Transition:} \quad s'_j = \sum_{i=0}^{n-1} s_i \cdot \phi(\mu_\sigma[i,j]) \quad \forall j \in [0,n) \tag{17}$$

$$\text{Initial boundary:} \quad s_i\big|_{t=0} = \phi(\alpha_i) \quad \forall i \in [0,n) \tag{18}$$

$$\text{Final boundary:} \quad \text{output} = \sum_{i=0}^{n-1} s_i\big|_{t=T} \cdot \phi(\eta_i) \tag{19}$$

## B.3 Proof

*Proof of Theorem 6.1.* We prove by induction on the number of input symbols processed that the state vector after processing $\sigma_1 \cdots \sigma_t$ satisfies $s_i|_t = \phi(v_i^{(t)})$ where $v^{(t)} = \alpha \otimes \mu_{\sigma_1} \otimes \cdots \otimes \mu_{\sigma_t}$.

**Base case** ($t = 0$): Boundary constraint (18) sets $s_i|_0 = \phi(\alpha_i) = \phi(v_i^{(0)})$. ✓

**Inductive step** ($t \to t+1$): Assume $s_i|_t = \phi(v_i^{(t)})$ for all $i$. By the transition constraint (17):

$$\begin{aligned}
s_j|_{t+1} &= \sum_{i=0}^{n-1} s_i|_t \cdot \phi(\mu_{\sigma_{t+1}}[i,j]) \\
&= \sum_{i=0}^{n-1} \phi(v_i^{(t)}) \cdot \phi(\mu_{\sigma_{t+1}}[i,j]) && \text{(inductive hypothesis)} \\
&= \sum_{i=0}^{n-1} \phi(v_i^{(t)} \otimes \mu_{\sigma_{t+1}}[i,j]) && (\phi \text{ is a semiring homomorphism}) \\
&= \phi\left(\bigoplus_{i=0}^{n-1} v_i^{(t)} \otimes \mu_{\sigma_{t+1}}[i,j]\right) && (\phi \text{ preserves } \oplus) \\
&= \phi(v_j^{(t+1)}) && \text{(WFA transition definition)} \quad ✓
\end{aligned}$$

**Final readout**: At $t = T = |w|$:

$$\text{output} = \sum_{i=0}^{n-1} \phi(v_i^{(T)}) \cdot \phi(\eta_i) = \phi\left(\bigoplus_{i=0}^{n-1} v_i^{(T)} \otimes \eta_i\right) = \phi([\![\mathcal{A}]\!](w)) \quad ✓$$

The key requirement is that $\phi$ is an injective semiring homomorphism (Definition E.1). Injectivity ensures that the field element $\phi([\![\mathcal{A}]\!](w))$ uniquely determines the semiring value $[\![\mathcal{A}]\!](w)$. □

# C Tier 2 Soundness Proof

## C.1 Comparison Gadget Soundness

**Lemma C.1** (Comparison gadget soundness)**.** *For $a, b \in [0, 2^{62})$ represented as Goldilocks field elements, the comparison gadget (Definition 6.2) correctly computes $\mathsf{lt}(a, b) \in \{0, 1\}$ and $\min(a, b)$ over $\mathbb{F}_p$.*

*Proof.* We verify each component:

**Difference-based comparison.** For $a, b \in [0, 2^{62})$, compute $d = b - a - 1$ over $\mathbb{F}_p$. If $a < b$: $d = b - a - 1 \in [0, 2^{62} - 2] \subset [0, 2^{62})$. If $a \geq b$: $d = b - a - 1 + p$ (wrapping in $\mathbb{F}_p$), giving $d \geq p - (2^{62} - 1) - 1 + 1 = p - 2^{62} + 1$. Since $p = 2^{64} - 2^{32} + 1 > 2^{63}$, we have $p - 2^{62} + 1 > 2^{63} - 2^{62} + 1 = 2^{62} + 1 > 2^{62}$.

**Bit decomposition as range check.** The constraints $d_i(d_i - 1) = 0$ for $i \in [0, 61]$ enforce $d_i \in \{0, 1\}$. The constraint $\sum_{i=0}^{61} d_i \cdot 2^i = d$ is satisfiable iff $d \in [0, 2^{62})$, since the maximum representable value with 62 bits is $2^{62} - 1$. Because $p > 2^{63} > 2^{62}$, there is no modular wraparound: the 62-bit representation of any $d \in [0, 2^{62})$ is unique.

Therefore: bit decomposition satisfiable $\Leftrightarrow d < 2^{62} \Leftrightarrow a < b$.

**Min selection.** Given correct $\mathsf{lt}(a, b)$:

$$\mathsf{lt}(a, b) \cdot a + (1 - \mathsf{lt}(a, b)) \cdot b = \begin{cases} a & \text{if } a < b \\ b & \text{if } a \geq b \end{cases} = \min(a, b) \quad \checkmark$$

□

**Lemma C.2** (Comparison gadget completeness)**.** *For every $a, b \in [0, 2^{62})$, there exists a unique valid assignment to the auxiliary columns of the comparison gadget.*

*Proof.* **Existence**: Given $a < b$, compute $d = b - a - 1 \in [0, 2^{62})$ and its binary representation $(d_0, \ldots, d_{61})$. This assignment satisfies all constraints by construction.

**Uniqueness**: Since $d \in [0, 2^{62})$ and $p > 2^{62}$, the 62-bit binary representation is unique (no modular wraparound). The less-than indicator and min-selector output are then determined. □

## C.2 Tropical Transition Encoding

A tropical WFA transition computes:

$$s'_j = \min_{i=0}^{n-1}(s_i + \mu_\sigma[i, j])$$

This is encoded using $n - 1$ comparison gadgets per state $j$. For $n$ states, this requires $(n - 1) \cdot n$ comparisons per row, adding $63 \cdot n \cdot (n - 1)$ auxiliary columns to the trace (62 bits for difference decomposition plus 1 indicator per comparison).

**Lemma C.3** (Tropical transition correctness)**.** *The sequence of comparison gadgets correctly computes $\min_{i=0}^{n-1}(s_i + \mu_\sigma[i, j])$ for each state $j$, provided all intermediate values remain in $[0, 2^{62})$.*

*Proof.* The minimum of $n$ values is computed iteratively: $m_1 = s_0 + \mu_\sigma[0, j]$, $m_{k+1} = \min(m_k, s_k + \mu_\sigma[k, j])$ for $k = 1, \ldots, n - 1$. By induction on $k$ and Lemma C.1, $m_k = \min_{i=0}^{k-1}(s_i + \mu_\sigma[i, j])$. The final value $m_n = \min_{i=0}^{n-1}(s_i + \mu_\sigma[i, j]) = s'_j$. □

### C.3 Full Tier 2 Proof

*Proof of Theorem 6.3.* By induction on input length, analogous to Theorem 6.1.

**Base case** ($t = 0$): Boundary constraints set $s_i|_0 = \alpha_i$ (initial weights). For tropical WFA, $\alpha_i \in \{0, +\infty\}$; we represent $+\infty$ as a sentinel value $M = 2^{61}$ (within the $[0, 2^{62})$ bound). All weights are offset-encoded as non-negative values: the ROUGE-L WFA uses negated match weights ($-1$ becomes $p - 1$ in the field), but the Tier 2 encoding instead uses non-negative weights ($+1$ for skip, 0 for match) so that minimization over non-negative values computes the negated LCS. This ensures all intermediate values remain in $[0, 2^{62})$.

**Inductive step**: Assume $s_i|_t$ correctly represents the tropical state vector. The Tier 2 transition constraints compute, for each $j$:

$$s_j|_{t+1} = \min_{i=0}^{n-1}(s_i|_t + \mu_{\sigma_{t+1}}[i, j])$$

By Lemma C.3, this is correct provided all intermediate values remain in $[0, 2^{62})$.

**Bound on intermediate values**: For a WFA with maximum weight $W_{\max}$ and input length $T$, intermediate values are bounded by $T \cdot W_{\max}$. We require $T \cdot W_{\max} < 2^{62}$, which holds for $T < 2^{62}/W_{\max}$. For NLP applications with $T \leq 10^4$ and $W_{\max} \leq 10^6$, this bound is satisfied with large margin.

**Final readout**: At $t = T$, the output column contains $\min_{i=0}^{n-1}(s_i|_T + \eta_i) = [\![\mathcal{A}]\!](w)$. □

**Remark on $+\infty$ representation.** The tropical semiring identity element $+\infty$ is represented as a finite sentinel $M = 2^{61}$. This is sound because $M$ exceeds any realizable path weight for inputs within the bound $T \cdot W_{\max} < 2^{61}$: the sentinel acts as "effectively infinite" since no real path can have weight $\geq M$. The domain restriction $[0, 2^{62})$ for the comparison gadget (Definition 6.2) is required because $p = 2^{64} - 2^{32} + 1 < 2^{64}$: a naïve 63-bit decomposition would incorrectly classify some $a \geq b$ cases as $a < b$ when $a - b > p - 2^{63}$. The condition $2^{k+1} \leq p$ (here $k = 62$, $2^{63} \leq p$) ensures the bit-decomposition range check is a faithful less-than test.

## D Boolean Embedding

**Proposition D.1** (Boolean embedding validity for deterministic WFA)**.** *Let $\mathcal{A}$ be a deterministic WFA over the Boolean semiring. The embedding $\phi : \{0, 1\} \to \mathbb{F}_p$ with $\phi(0) = 0$, $\phi(1) = 1$ correctly encodes all WFA transitions under field arithmetic.*

*Proof.* The Boolean embedding fails in general because $\phi(1 \vee 1) = \phi(1) = 1 \neq 2 = \phi(1) + \phi(1)$. This occurs when two or more incoming transitions contribute weight 1 to the same state, and the field sum exceeds 1.

In a deterministic WFA, for each state $q_j$ and input symbol $\sigma$, at most one predecessor state $q_i$ has a non-zero transition weight $\mu_\sigma[i, j] = 1$. Therefore, in the transition sum:

$$s'_j = \bigoplus_{i=0}^{n-1} v_i \otimes \mu_\sigma[i, j] = \bigvee_{i=0}^{n-1} (v_i \wedge \mu_\sigma[i, j])$$

at most one term is non-zero. Consequently:

$$\phi(s'_j) = \sum_{i=0}^{n-1} \phi(v_i) \cdot \phi(\mu_\sigma[i, j])$$

where the sum has at most one non-zero term. The field sum equals 0 or 1, matching $\phi(s'_j)$ exactly.

For the exact-match WFA specifically: state $q_k$ is active (weight 1) iff the first $k$ tokens match. At each step, either $q_k$ transitions to $q_{k+1}$ (match) or to a sink state (mismatch), with exactly one transition taken. No state ever receives contributions from multiple predecessors. □

# E Semiring Axiom Verification

## E.1 Semiring Definitions

Table 9: Semirings supported by SPECTACLES.

| **Name** | $\mathcal{S}$ | $\oplus$ | $\otimes$ | $\bar{0}$ | $\bar{1}$ |
|---|---|---|---|---|---|
| Boolean | $\{0,1\}$ | $\vee$ | $\wedge$ | 0 | 1 |
| Counting | $\mathbb{N}$ | $+$ | $\cdot$ | 0 | 1 |
| Tropical | $\mathbb{Z} \cup \{+\infty\}$ | min | $+$ | $+\infty$ | 0 |
| Goldilocks | $\mathbb{F}_p$ | $+_p$ | $\cdot_p$ | 0 | 1 |

**Definition E.1** (Embeddable semiring)**.** *A semiring* $(\mathcal{S}, \oplus, \otimes, \bar{0}, \bar{1})$ *is* $\mathbb{F}_p$-embeddable *if there exists an injective map* $\phi : \mathcal{S} \to \mathbb{F}_p$ *satisfying* $\phi(\bar{0}) = 0$*,* $\phi(\bar{1}) = 1$*,* $\phi(a \oplus b) = \phi(a) + \phi(b)$*, and* $\phi(a \otimes b) = \phi(a) \cdot \phi(b)$*.*

**Proposition E.2** (Counting semiring embeddability)**.** *The counting semiring* $(\mathbb{N}, +, \cdot, 0, 1)$ *embeds into* $\mathbb{F}_p$ *for* $p = 2^{64} - 2^{32} + 1$ *via the canonical inclusion* $\phi(n) = n \bmod p$*, which is injective for* $n < p$*.*

*Proof.* For $n, m \in \mathbb{N}$ with $n, m < p$ and $n+m < p$: $\phi(n+m) = (n+m) \bmod p = n+m = \phi(n)+\phi(m)$. Similarly for multiplication. For NLP applications, all intermediate counting values are bounded by input length (at most $\sim 10^4$), well below $p \approx 1.8 \times 10^{19}$. □

## E.2 Verification Methodology

Semiring axioms are verified at three levels:

**Lean 4 formalization.** The `KleeneSemiring` typeclass formalizes the eight core axioms (commutativity, associativity, distributivity of $\otimes$ over $\oplus$ from both sides, left and right identity for $\oplus$ and $\otimes$, and zero annihilation from both sides). All semiring axiom proofs for Boolean, counting, and Goldilocks types are machine-checked and sorry-free.

**Property-based testing.** Each axiom is tested via Rust `proptest` with 256 random inputs per semiring type. The eight axioms tested are:

1. $a \oplus b = b \oplus a$ (additive commutativity)
2. $(a \oplus b) \oplus c = a \oplus (b \oplus c)$ (additive associativity)
3. $a \oplus \bar{0} = a$ (additive identity)
4. $(a \otimes b) \otimes c = a \otimes (b \otimes c)$ (multiplicative associativity)
5. $a \otimes \bar{1} = a = \bar{1} \otimes a$ (multiplicative identity)
6. $a \otimes (b \oplus c) = (a \otimes b) \oplus (a \otimes c)$ (left distributivity)
7. $(a \oplus b) \otimes c = (a \otimes c) \oplus (b \otimes c)$ (right distributivity)

8. $a \otimes \bar{0} = \bar{0} = \bar{0} \otimes a$ (zero annihilation)

Total: 125 unit tests across all semiring types.

**Python cross-validation.** An independent Python test suite re-verifies all axioms with 22 test functions and 500 random trials per axiom, providing cross-language validation.

# F PSI Security Analysis

## F.1 Commit-Then-Execute Framework

The PSI protocol operates in a *commit-then-execute* framework designed to limit the deviation space of a potentially malicious party:

1. **Commitment phase**: Both parties independently commit to their $n$-gram sets by publishing a binding commitment $C_i = H(\text{nonce}_i \| L_i)$ using BLAKE3.
2. **Verification**: Each party proves that their PSI inputs match their committed set by revealing the nonce and allowing the counterparty to recompute the commitment.
3. **PSI execution**: The OPRF-based PSI protocol executes on the committed inputs under the semi-honest model.

## F.2 Security Argument

The semi-honest PSI assumption is commonly criticized as inadequate for adversarial settings. The commit-then-execute framework provides concrete defenses:

**Input substitution attack.** A malicious evaluator might substitute a clean $n$-gram set to hide contamination. The commitment binding prevents this: after committing $C = H(\text{nonce} \| L)$, the evaluator cannot produce $L' \neq L$ that satisfies $C = H(\text{nonce}' \| L')$ without breaking BLAKE3 collision resistance ($2^{-128}$ security).

**Selective revelation.** A malicious data provider might selectively omit $n$-grams to reduce apparent overlap. Commitment binding prevents omission: the full committed set must be used.

**Residual limitations.** The commit-then-execute framework does *not* defend against:

- A party who commits to an *intentionally incomplete* $n$-gram extraction (e.g., extracting $n$-grams from only a subset of training data). This requires trust in the extraction process, not the protocol.
- Paraphrase memorization, which produces no $n$-gram signal regardless of protocol security.
- A fully malicious party who deviates from the OPRF protocol itself. Upgrading to malicious-secure PSI [8] would address this at the cost of higher communication complexity.

No UC security proof [5] exists for the composition of STARK proofs with the PSI protocol. The overall system provides a concrete security argument, not a simulation-based proof.

# G Full Benchmark Data

## G.1 Detailed Benchmark Scores

## G.2 Compilation Correctness Breakdown

The 57,518 compilation correctness checks comprise:

Table 10: Full benchmark evaluation results.

| Category | Pairs | EM | Tok. F1 | BLEU | R-1 | R-L | Disagree. |
|---|---|---|---|---|---|---|---|
| MMLU-style | 30 | 0.600 | 0.600 | 0.936 | 0.600 | 0.600 | 0 |
| SQuAD-style | 20 | 0.050 | 0.602 | 0.321 | 0.602 | 0.582 | 0 |
| Translation | 15 | 0.000 | 0.625 | 0.364 | 0.625 | 0.602 | 0 |
| Random stress | 500 | 0.002 | 0.121 | 0.218 | 0.121 | 0.110 | 0 |
| **Total** | **565** | — | — | — | — | — | **0** |

- 50,750 triple-agreement checks (10 seeds $\times$ 5 metrics $\times$ 1,015 pairs)
- 5,075 score-range validity checks ($\forall$ pair, metric: $0 \leq \text{score} \leq 1$)
- 1,524 reflexivity checks ($\text{EM}(x, x) = 1$, $\text{F1}(x, x) = 1$, $\text{BLEU}(x, x) \in [0, 1]$)
- 169 edge-case checks (8 cases $\times$ 10 seeds, plus cross-metric)

Property-based tests additionally verify:

- 2,600 triple-agreement property checks ($\forall$ pair, metric: ref = aut = cir, all passed)
- 2,600 score-range property checks (all passed)
- 520 exact-match reflexivity checks ($\forall s$: $\text{EM}(s, s) = 1$, all passed)
- 520 token-F1 reflexivity checks ($\forall s \neq \varepsilon$: $\text{F1}(s, s) = 1.0$, all passed)
- 520 BLEU reflexivity checks ($\forall s$ with $\geq 4$ tokens: $\text{BLEU}(s, s) \approx 1.0$, all passed)

### G.3 STARK Proof Detailed Data

The WFA simulation circuits demonstrate scaling from 4 to 128 effective states, with proving time growing approximately linearly. The $n$-gram counting circuits model the core operation of Token F1 and ROUGE metrics. All 21 proofs verify at 128-bit security.

## H Ablation Analysis

We analyze the marginal contribution of each verification component by comparing the full system against a version with that component removed (Table 12).

**Key finding**: Triple verification is the highest-impact component. Both implementation bugs found during development were in the circuit code path (Goldilocks field arithmetic). Dual verification (reference vs. WFA) would not exercise circuit code and would miss these bugs. The WFA equivalence checker provides a fundamentally new capability: deciding whether two metric implementations agree on *all* inputs, not just a finite test set.

## I Sorry Audit

The Lean 4 formalization contains 17 `sorry` placeholders, categorized by difficulty, effort to close, and position in the proof dependency graph.

### I.1 Routine Sorrys (12)

These are targetable by standard Lean tactics (`omega`, `decide`, `simp`, `norm_num`); estimated total effort: $\sim$1 week.

Table 11: Complete STARK proof measurements including scaling benchmarks.

| **Circuit** | **States** | **Prove (ms)** | **Verify (ms)** | **Size (KiB)** | **Valid** |
|---|---|---|---|---|---|
| fibonacci_8 | 2 | 3.9 | 0.3 | 35.6 | ✓ |
| fibonacci_16 | 2 | 8.9 | 0.4 | 55.8 | ✓ |
| fibonacci_32 | 2 | 5.2 | 0.5 | 70.7 | ✓ |
| fibonacci_64 | 2 | 10.6 | 0.7 | 86.8 | ✓ |
| counter_8 | 1 | 6.8 | 0.2 | 35.3 | ✓ |
| counter_16 | 1 | 5.7 | 0.4 | 55.5 | ✓ |
| counter_32 | 1 | 6.7 | 0.5 | 70.4 | ✓ |
| counter_64 | 1 | 4.8 | 0.7 | 86.5 | ✓ |
| wfa_sim_4_states | 4 | 11.0 | 0.4 | 56.5 | ✓ |
| wfa_sim_8_states | 8 | 5.5 | 0.4 | 57.8 | ✓ |
| wfa_sim_16_states | 16 | 8.9 | 0.4 | 60.3 | ✓ |
| wfa_sim_32_states | 32 | 20.3 | 0.6 | 80.3 | ✓ |
| wfa_sim_48_states | 48 | 44.6 | 0.7 | 101.5 | ✓ |
| wfa_sim_64_states | 64 | 65.4 | 1.4 | 106.6 | ✓ |
| wfa_sim_96_states | 96 | 169.4 | 0.9 | 134.1 | ✓ |
| wfa_sim_128_states | 128 | 198.4 | 1.0 | 144.4 | ✓ |
| ngram_count_len_16 | 5 | 12.7 | 0.4 | 55.8 | ✓ |
| ngram_count_len_32 | 5 | 4.5 | 0.5 | 70.7 | ✓ |
| ngram_count_len_64 | 5 | 9.1 | 0.7 | 86.8 | ✓ |
| ngram_count_len_128 | 5 | 6.5 | 0.8 | 104.0 | ✓ |
| ngram_count_len_256 | 5 | 13.8 | 1.0 | 122.5 | ✓ |
| **Total** | — | — | — | — | 21/21 |

1. Field commutativity instantiations (3 sorrys): specific instances of $a \cdot_p b = b \cdot_p a$ for Goldilocks field elements. *Tactic*: `ring` or `omega`. Evidence: 3,500 random tests pass.
2. Trivial inequality lemmas (4 sorrys): bounds of the form $n < p$ where $n$ is a small constant. *Tactic*: `norm_num`. Evidence: trivially true ($n \leq 10^4$, $p \approx 1.8 \times 10^{19}$).
3. Identity simplifications (3 sorrys): lemmas of the form $a + 0 = a$ or $a \cdot 1 = a$ in specific proof contexts. *Tactic*: `simp [add_zero, mul_one]`. Evidence: exhaustively tested.
4. Decidability instances (2 sorrys): decidable equality for specific semiring element types. *Tactic*: `derive` or manual instance. Evidence: Boolean (2 values) and $\mathbb{N}$ (`Nat.decEq`).

None of these routine sorrys are on the critical path for any novel theorem. They represent proof engineering debt, not mathematical uncertainty.

### I.2 Novel Sorrys (5)

These require new proof techniques; estimated total effort: ∼6–10 weeks.

1. **Hopcroft minimization** (1 sorry, ∼2–4 weeks): The Hopcroft partition refinement algorithm [10] produces a minimal DFA. The sorry asserts that the partition refinement loop terminates and produces equivalent states. This is a well-known result but requires careful Lean encoding of the refinement invariant. *Evidence*: brute-force enumeration validates equivalence for all tested WFA pairs (strings up to length 20, >1M strings). *Reference*: Lammich and Tuerk (2012) formalized this in Isabelle/HOL.

Table 12: Ablation analysis: contribution of each verification component.

| Component | Marginal Contribution | Verdict |
|---|---|---|
| Triple verification (ref+WFA+circuit) | +2 real bugs found (Montgomery constant, Lagrange interpolation) | **Essential**: both bugs were in the circuit path and would be missed by dual (ref+WFA) verification |
| Lean 4 formalization | Mathematical certainty for 8 semiring axioms; machine-checked theorem statements | **Valuable**: provides formal foundations but does not replace testing for implementation correctness |
| Tier 2 compilation | Adds ROUGE-L (tropical semiring) | **Needed for completeness**: covers 1/6 metrics at significant complexity cost (quadratic trace width) |
| Contamination detection | Privacy-preserving $n$-gram overlap check | **Complementary**: addresses data integrity gap but is a proof-of-concept |
| Property-based testing | 9,839 structured axiom tests with shrinking | **Valuable**: provides algebraic coverage beyond random I/O testing |
| WFA equivalence | Universal specification equivalence on $\forall w \in \Sigma^*$ | **Unique capability**: no other NLP eval tool provides specification-level equivalence checking |

2. **WFA-to-AIR compilation** (2 sorrys, ∼1–2 weeks each): The compilation function from WFA transition matrices to AIR polynomial constraints assumes (i) correct trace layout assignment and (ii) constraint polynomial degree bounds. These are structural properties of the compiler output. *Evidence*: 57,518 differential tests with 0 disagreements; 21/21 STARK proofs verify (FRI would reject degree-violating constraints).
3. **Tier 2 gadget composition** (1 sorry, ∼1 week): The composition of multiple comparison gadgets in a single trace row assumes non-interference between gadget auxiliary columns. This is enforced by the column allocation scheme but not yet proved. *Evidence*: 10,150 ROUGE-L differential test pairs with 0 disagreements; column allocator uses monotonically increasing offsets.
4. **Kleene star convergence for tropical** (1 sorry, ∼3–5 days): The Kleene star $a^* = \bigoplus_{n=0}^{\infty} a^{\otimes n}$ converges in the tropical semiring only when $a \geq 0$ (under min-+ convention). The sorry assumes this convergence condition. *Evidence*: mathematically trivial ($\min(0, a, 2a, \ldots) = 0$ for $a \geq 0$); tested in Rust and Python. This sorry is *not* on the critical path (ROUGE-L does not use the Kleene star).

**Critical path analysis**: No sorry contaminates the core semiring axiom proofs, which are fully

machine-checked. The compilation soundness theorem (Theorem 6.1) is stated in Lean but its proof chain includes the WFA-to-AIR sorrys. The theorem is empirically verified by 57,518 differential tests.

**Dependency analysis**: The 5 novel sorrys have the following blast radii: (i) The Hopcroft sorry affects only the WFA minimization decision procedure; equivalence results remain valid via brute-force enumeration up to length 20. (ii) The two WFA-to-AIR sorrys affect the proof chain for Theorem 6.1, making the compilation soundness formally *stated but not proved*; the 57,518 differential tests provide empirical confidence. (iii) The gadget composition sorry affects Tier 2 soundness (Theorem 6.3) for tropical circuits only. (iv) The Kleene star convergence sorry affects tropical WFA closure operations, not the main evaluation pipeline. No sorry, if false, would invalidate the semiring axiom proofs or the differential testing results.

# J Lean–Rust Formal Correspondence

The Lean 4 formalization and Rust implementation are separate codebases. Table 13 maps key Lean definitions to their Rust counterparts.

Table 13: Lean 4 to Rust correspondence with test coverage.

| **Lean Definition** | **Rust Implementation** | **Bridge** |
|---|---|---|
| `class KleeneSemiring` | `trait Semiring` | 9,839 property tests |
| `instance BoolSR` | `impl Semiring for Bool` | Exhaustive (40 tests) |
| `instance NatSR` | `impl Semiring for Counting` | 2,500 random tests |
| `instance GoldilocksSR` | `GoldilocksField ops` | 3,500 random tests |
| `embed_bool_hom` | `BooleanEmbedding` | Exhaustive (8 tests) |
| `embed_count_hom` | `CountingEmbedding` | 1,000 random tests |
| `theorem tier1_sound` | `compile_algebraic()` | 50,750 diff. tests |
| `theorem tier2_sound` | `compile_gadget_assisted()` | ROUGE-L diff. tests |
| `wfa_equiv_decidable` | `WFAEquivalenceChecker` | Equiv. property tests |

The bridge is empirical, not machine-checked. This is the largest verification gap in the system. Verified extraction (generating Rust from Lean specifications) would close this gap but is beyond current Lean 4 tooling for Rust targets.